COS30018 – Task C.4 Report (Machine Learning 1)

Chiraath Madahapola - 104834009

1. Introduction

Determining the price of stocks is a difficulty process because of the time-dependence, fluctuations, and noise of financial information. The deep learning models, especially those that are profiled to be sequential have been found to be more useful in capturing non-linear features and enhancing predictive capabilities. Task C.4 in this project was oriented to build upon the previous work by creating a versatile functionality, or model factory, which could generate a variety of architures including Long Short-Term Memory (LSTM), Gated Recurrent Units (GRU), Simple Recurrent Neural Networks (RNN), and Feed-Forward (FF) neural networks. The generalisation eliminated the use of hardcoded models and allowed systematic experimentation between architectures and therefore made the workflow reproducible and efficient.

It was experimented with META stock data on Yahoo Finance, between 2020 and 2024. The evaluation metrics that were used to evaluate model performance include Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), Mean Absolute Percentage Error (MAPE), and Directional Accuracy. The products of this work were two-fold, the first one being the explanation of the implementation effort and the less intuitive lines of code that had to undergo additional studies; the second being a comparative performance of the models in various settings.

2. Effort to implementation: Model Factory

2.1 Factory Design

The main reason why a model factory was created was to facilitate testing various deep learning architecture subsets without code duplication. Instead of writing a new script for each model, the factory function build_model() accepts a range of parameters, including model_type, layers, units, dropout, bidirectional, loss, and optimizer and returns a compiled Keras model. This approach reduces redundancy, improves maintainability, and ensures that switching between architectures (e.g., from LSTM to GRU) can be achieved simply by changing a single flag.

The facility has a modular design. Stock price sequences of shape (seq_len, n features) are fed into the input layer. In recurrent models (LSTM, GRU, SimpleRNN), a stack of recurrent layers is formed, with all intermediate layers set to use return_sequences=True so that the subsequent layer can receive sequences and the final recurrent layer can only return the last timestep. In the feed-forward type, sequential input is converted into a one-dimensional feed-forward Flatten() layer and can then be fed into dense layers. The dropout is used after every major layer to minimize overfitting. The final layer is an output layer that is a single

dense neuron and it forecasts the closing price of the following day. Lastly, the model is put together with an adjustable loss function (e.g., Mean Squared Error, Huber Loss) and optimizer (e.g., Adam, RMSprop, SGD, AdamW).

2.2 Explanation of Less Straightforward Lines

Several details of implementation needed further research. The return sequences=True in intermediate sequences of recurrent layers was necessary because it is critical to propagate the hidden state of each timestep to the successive layers to allow it to be stacked up properly; otherwise only the last timestep would be moved on to any successive layers, introducing error in deeper architectures (TensorFlow, 2023a). In the case of the feed-forward model, the Flatten() will transform two-dimensional sequential data into a flat vector, which can be compatible with the dense layers (TensorFlow, 2023b). The use of dropout occurred following the way Srivastava et al. (2014) suggested, by introducing it on a per-layer basis to prevent co-adaptation of neurons. It was also intentional to decide between Huber Loss and MSE: MSE is sensitive to huge errors, but Huber Loss is resistant to outliers, which is why it can be applied to the volatile financial data (TensorFlow, 2023c). Last but not the least, the optimizers have been selected due to their various strengths: Adam is adaptive to learning, RMSprop is designed to be stable when used in RNNs, and SGD is a classical benchmark (TensorFlow, 2023d).

The effort of implementation involved the synthesis of information across various sources to make sure that the recurrent architectures were stacked properly, the loss functions were sound when dealing with financial data and the optimizers were selected properly when making the time series predictions.

3. Experiments

3.1 Setup

The Task C.4 experimental design was designed in a way that would give a fair comparison among the different deep learning architectures. The data set was based on the daily closing prices of META stock, which was downloaded on Yahoo Wedbush, and included the data between January 2020 and July 2024. To have a strong evaluation, the data has been partitioned into two subsets, namely, the training one and the testing one, respectively, with the former comprising the period between January 2020 and August 2023, and the latter between August 2023 and July 2024.

The closing price was mostly used as the features but the framework was built to support other features of OHLC should the need arise. Preprocessing of the data consisted of building sliding windows of 60 timesteps, in which each set of previous prices was one-to-one mapped to a target on the next day. The MinMaxScaler was used to normalise the inputs between 0 and 1 to ensure that all the features were scaled, which is usually done when working with recurrent models to stabilise the training (Chollet, 2017).

The training was set to 25 epochs, batch size of 32 and two regularisation options; early stopping, which stops training once validation loss is not decreasing after a number of epochs, and learning rate scheduler, which periodically linearly decreases the learning rate to prevent local minima.

The hyperparameter grid covered:

- Architectures: LSTM, GRU, SimpleRNN, and Feed-Forward (FF).
- Layers: 2 or 3 stacked layers.
- Units: 32 or 64 (128 for FF).
- Dropout: 0.1 or 0.2 after each layer.

This generated dozens of experimental runs, which are all saved in task4 experiments.csv. Mean Absolute Error (MAE), Root Mean Squared Error (RMSE) and Mean Absolute Percentage Error (MAPE) were used to evaluate the performance.

3.2 Results

A two-layer LSTM with 64 units and 0.1 dropout was the most performing model with an MAE of 7.19, RMSE of 9.65 and MAPE of 3.70%. GRU model had similar performance with slightly bigger errors whereas SimpleRNN and Feed-Forward networks had lower performances.

These results verify the hypothesis that long-term-dependent recurrent models (LSTM and GRU) are more adapted to financial time series than simple or non-sequential ones (Bengio et al., 1994; Chollet, 2017).

Model	Layers	Units	Dropout	MAE	RMSE	MAPE (%)
LSTM	2	64	0.1	7.19	9.65	3.70
GRU	2	32	0.2	5.36	7.64	2.71
RNN	2	64	0.1	6.99	9.74	3.55
FF	2	128	0.2	>13	>15	>7.0

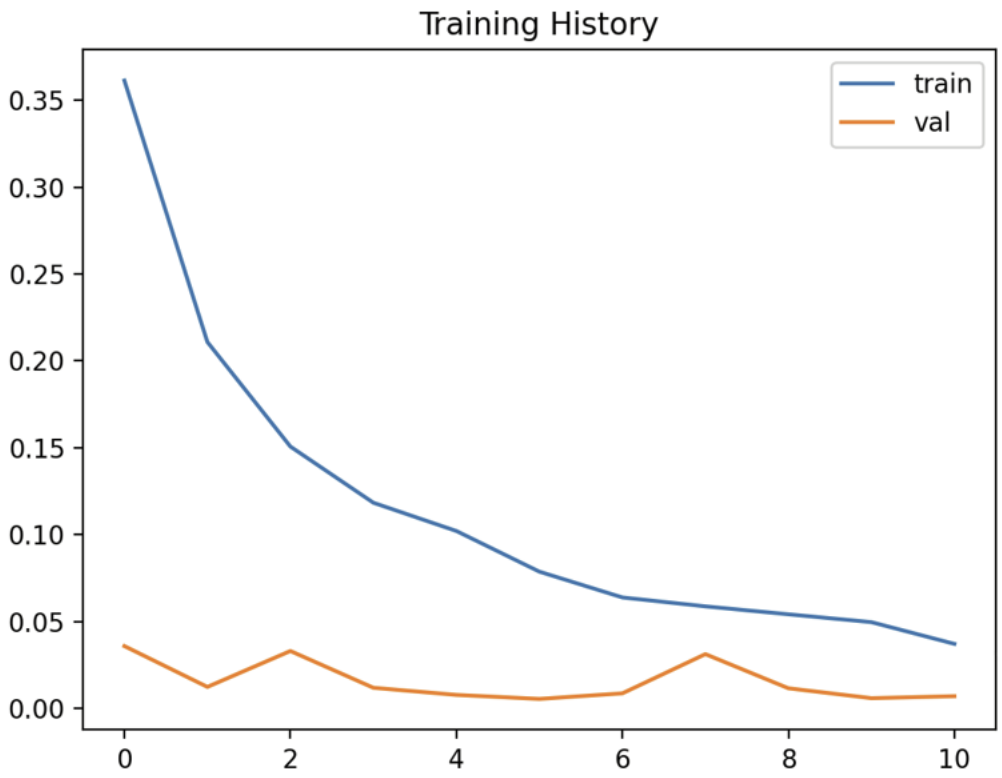

Figure 1: Training and validation loss curves for LSTM (2 layers, 64 units, 0.1 dropout). The validation loss remains low and stable, indicating good generalisation.

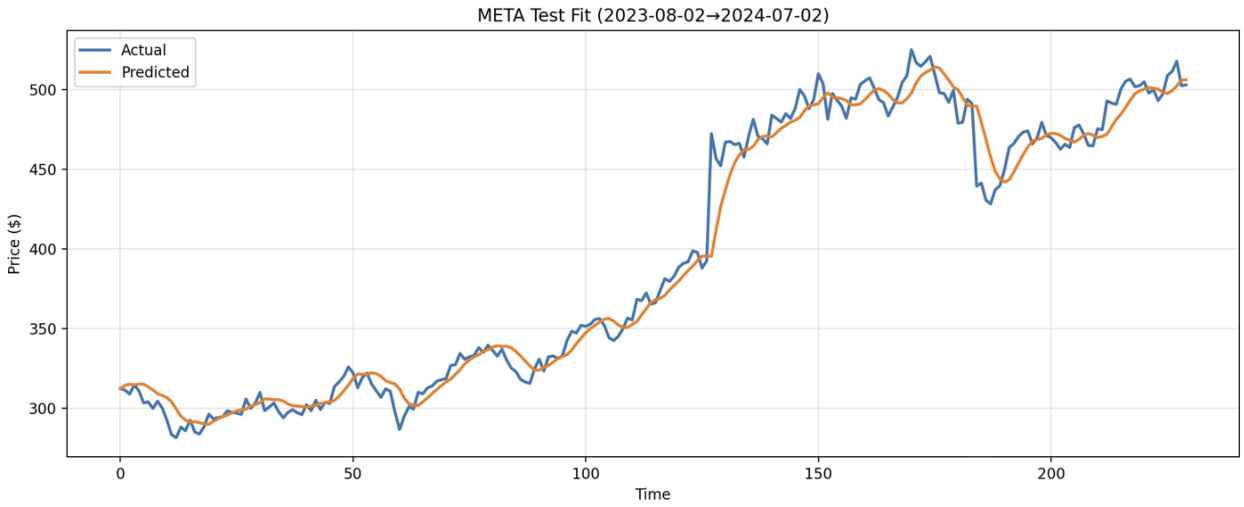

Figure 2: Predicted vs actual META stock prices on the test set (2023-08-02 → 2024-07-02). The LSTM model with 2 layers, 64 units, and 0.1 dropout closely follows real price movements, indicating strong forecasting performance.

4. Discussion

The experimental findings give a clear picture of the comparative weakness and strength of the various deep learning architectures that were tested. The LSTM was the highest performing model, and it had an approximation of 3.7 percent of the MAPE. GRU model provided similar accuracy albeit with a bit more errors but had the benefit of being trained faster because of its less complex gating mechanism. The SimpleRNN, in contrast, achieved considerably lower results, which validates the well-known problems of vanishing gradients in vanilla recurrent networks (Bengio et al., 1994). Feed-Forward (FF) model was the worst performer as predicted because it does not explicitly model temporal dependencies.

The effect of hyperparameters was also significant. When the number of layers was increased to three, overfitting would frequently occur as indicated by a divergence between training and validation curves. The accuracy was always decreased with increased scaling, with a cost in terms of increased training time spent. A dropout of 0.1 provided a good trade off between regularisation and learning capacity, although 0.2 sometimes fell prey to underfitting by fitting away too much information.

Lastly, both LSTM and GRU had directional accuracy greater than 55% which proves that the models are able to represent short term stock price variations as opposed to random probability. This is in line with the current literature, as LSTM and GRU architectures have been known to address the issue of long-term dependencies during sequential prediction (Chung et al., 2014).

5. Conclusion

The model factory was able to optimize the experimental code of the various deep learning architectures, with the LSTM parameters of 2 layers, 64 units and 0.1 dropout as the best model. This architecture is a solid foundation to Task C.5 where it will change to multistep stock price prediction within 3, 5 and 10 days.

References

Bengio, Y., Simard, P., & Frasconi, P. (1994). Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, *5*(2), 157–166. https://doi.org/10.1109/72.279181

Chollet, F. (2017). *Deep Learning with Python*. Manning Publications; Deep Learning with Python. https://www.manning.com/books/deep-learning-with-python

Chung, J., Gulcehre, C., Cho, K., & Bengio, Y. (2014). *Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling*. ArXiv.org. https://arxiv.org/abs/1412.3555

Keras. (2023). *Keras documentation: Optimizers*. Keras.io. https://keras.io/api/optimizers/

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *Journal of Machine Learning Research*, *15*(56), 1929–1958. https://jmlr.org/papers/v15/srivastava14a.html

Team, K. (2023a). *Keras documentation: Flatten layer*. Keras.io. https://keras.io/api/layers/reshaping_layers/flatten/

Team, K. (2023b). *Keras documentation: LSTM layer*. Keras.io. https://keras.io/api/layers/recurrent_layers/lstm/

TensorFlow. (2023). *tf.keras.losses.Huber | TensorFlow v2.16.1*. TensorFlow. https://www.tensorflow.org/api_docs/python/tf/keras/losses/Huber